

Top Design Patterns for Interviews

Frequency Ratings — Saudi | Dubai | Malaysia | India Tier-2

Interview Prep Reference

🔗 Top Design Patterns Asked in Interviews

Frequency Ratings — Saudi Arabia | Dubai/UAE | Malaysia | India Tier-2

How to Use This Document

Design pattern rounds almost never ask “*explain the Observer pattern*” in isolation. Instead, you get a **business problem** (Parking Lot, Ride-Hailing, Notification System) and are expected to **derive the right pattern(s)** while designing classes. This guide ranks patterns by how often they show up **in practice**, across your four target markets, so you spend prep time where it actually pays off.

Rating scale: 🟢 Very High · 🟡 High · 🟠 Medium · 🟤 Low

1. Master Frequency Table

#	Pattern	Category	Frequency	Where It Shows Up
1	Strategy	Behavioral	🟢 Very High	Pricing/discount engines, payment gateways, sorting/eviction policies — appears in almost every LLD question
2	Factory / Abstract Factory	Creational	🟢 Very High	Object creation in Parking Lot, Vehicle systems, Notification channels
3	Singleton	Creational	🟢 Very High	Logger, Config Manager, DB Connection Pool — common warm-up question
4	Observer	Behavioral	🟢 Very High	Notification systems, stock tickers, event-driven/pub-sub systems
5	Decorator	Structural	🟡 High	Pizza/Coffee ordering, middleware chains, UI customization

6	Builder	Creational	⦿ High	Complex object construction — meal ordering, HTTP request builder, query builder
7	State	Behavioral	⦿ High	Elevator system, Vending Machine, Order lifecycle (Pending → Shipped → Delivered)
8	Adapter	Structural	⦿ Medium	Third-party API integration, wrapping legacy systems
9	Command	Behavioral	⦿ Medium	Undo/Redo systems, Task Scheduler, Remote Control design
10	Chain of Responsibility	Behavioral	⦿ Medium	Middleware/filters, approval workflows, logging levels
11	Composite	Structural	⦿ Medium	File System design, Org hierarchy, UI tree structures
12	Facade	Structural	⦿ Low-Medium	Simplifying subsystem access — mentioned, rarely deep-dived
13	Proxy	Structural	⦿ Low-Medium	Caching layer, Rate limiter, Lazy loading
14	Template Method	Behavioral	⦿ Low	Report generation, algorithm skeletons
15	Prototype	Creational	⦿ Low	Object cloning, invoice/template duplication — asked standalone occasionally

Prep priority: If time is short, master rows 1-7 in depth (code + trade-offs). Rows 8-11 need working knowledge. Rows 12-15 need definition + one example — enough to not go blank.

2. Classic LLD Problems → Patterns Tested

Interviewers reuse a fairly small set of “design X” prompts across companies. Recognizing the prompt should immediately trigger the pattern list below.

LLD Problem	Patterns Tested
Parking Lot System	Factory, Strategy, Singleton
Elevator System	State, Strategy, Observer
Splitwise / Expense Sharing	Strategy, Observer
Movie Ticket Booking (BookMyShow-style)	Factory, Singleton, State, Strategy
Food Delivery	Observer, Strategy, State,

(Swiggy/Zomato/Talabat-style)	Factory
Ride-Sharing (Uber/Careem-style)	Strategy, Observer, Factory, State
Rate Limiter	Strategy, Proxy, Singleton
Notification System (Email/SMS/Push)	Observer, Factory, Strategy
Vending Machine	State, Strategy
Chess / Tic-Tac-Toe	State, Strategy, Command
Logging Framework	Singleton, Chain of Responsibility, Strategy
LRU Cache Design	Strategy (eviction policy), Singleton
ATM Machine	State, Strategy, Command
Library Management System	Factory, Observer, Singleton
Hotel Booking System	Strategy, State, Factory, Observer

3. Recognition Triggers (Pattern-Spotting Cheat Sheet)

Use this the way you'd use a DSA "if you see X, think Y" trigger card.

If the problem says...	Think...
"Multiple ways to calculate/process something (payment, discount, route)"	Strategy
"Object creation should be centralized/hidden from client"	Factory
"Only one instance should ever exist (config, logger, connection pool)"	Singleton
"Notify multiple parts of the system when something changes"	Observer
"Add optional features/behavior without changing the base class"	Decorator
"Object has many optional fields / complex step-by-step construction"	Builder
"Behavior changes based on internal status/lifecycle"	State
"Integrate with a third-party API that has a different interface"	Adapter
"Need undo, redo, or queued/deferred actions"	Command
"Request passes through multiple handlers (validation, auth, logging)"	Chain of Responsibility
"Tree-like structure — folders/files, org chart, menus"	Composite
"Simplify a complex subsystem behind one clean interface"	Facade
"Control/restrict access to an object (caching, lazy-load, permissions)"	Proxy
"Object creation is expensive and you need many similar copies"	Prototype

4. Regional Notes

Saudi Arabia / Dubai (Careem, Talabat, Noon, banks & fintech)

Heavy emphasis on **Strategy + Observer + Factory** combos, almost always wrapped in a real business domain (ride-hailing, delivery, payments) rather than asked as an abstract quiz. Interviewers want you to **derive** the pattern from requirements, not recite a definition. Fintech/banking interviews in this region also lean on **State** (transaction lifecycle) and **Chain of Responsibility** (fraud/approval pipelines).

Malaysia (Grab-style, fintech, telco)

Similar core set — **Strategy / State / Observer** dominate — but often paired with **concurrency concerns** (thread-safety in Singleton, idempotency in Command). Expect the interviewer to push on “what happens under concurrent access” after you present the class diagram.

India Tier-2

More traditional and syllabus-driven. Expect **direct definitional questions** (“explain Singleton vs Factory”, “implement Observer in Java/PHP”) *alongside* one full LLD problem. **Parking Lot** and **Elevator System** are extremely common here as the “full design” round.

5. How Interviewers Actually Test This (Across All Regions)

Requirements → Identify entities → Class diagram → Code → Pattern justification

The pattern name is rarely the point — **the ability to justify why Strategy fits here instead of an if-else chain** is what’s actually being scored. A strong answer always includes the trade-off: *“I used Strategy instead of a switch statement because new payment methods will be added frequently, and this keeps the Order class closed for modification.”*

6. Quick Self-Check Before Any LLD Round

- ☐ Can I state Strategy, Factory, Singleton, Observer definitions in under 15 seconds each?
- ☐ Can I sketch Parking Lot + Elevator + Notification System class diagrams from memory?
- ☐ For each pattern, do I have **one real production example**, not just a textbook one?
- ☐ Can I explain thread-safety for Singleton (lazy vs eager init, double-checked locking)?
- ☐ Can I explain why I’d choose Strategy over a simple if-else, out loud, unprompted?

Companion reference: see the dedicated Prototype Pattern deep-dive guide for the full-depth treatment (production examples, shallow vs deep copy, Q&A bank) — apply the same depth to Strategy, Factory, Observer, and Singleton next, since those are your 🌀 Very High priority patterns.